

advanced-programming-group-runge-kutta

Task Allocation

Hazem Bakhshwain	Git-Hub Manager, Task Allocation, Image base class, Volume base class, Greyscale, Slicing, main and args parser, result comparison with ImageMagic, and general bug fixing.
Arastun Mammadli	Salt-pepper noise filter, 2D Gaussian blur filter, 3D blur filters (Median, Gaussian), optimisation of 2D and 3D blur filters, instructions README, works on documentation (using Doxygen).
Yiding Hao	Brightness, Sharpening filter, Projection, compare the results of our project with opencv. Helped with report. Debugged multiple blurs and filters.
Chenwei Yin	Edge Detection filter, writing report, helped Yiding with Projection, debugged some filters and 3D functions.
Tingting Chen	Medianblur filter, write all unit tests with Yixuan Yan.
Yixuan Yan	Histogram equalization filter, finish all tests with Tingting Chen, helped with report and references .
Adityee Jain	Threshold, box blur filter, and image base class helper functions.

Project classes structure:

The project has been structured in these files: main.cpp, Image.h.cpp, Volume.h.cpp, Filter.h.cpp, Projection.h.cpp, and Slicing.h.cpp. The Image class and Volume class is written to bind all the project files together by using a base framework for input image or volume, as well as the interface functions that give other files the access to modify the values in the image or volume. In each filtering, blurring, and 3D function, the data of image or volume can be accessed and modified, therefore, the classes could interact through the interfaces implemented in the image and volume classes. The blurring and filtering functions have been implemented in the Filter.h.cpp, projection and slicing functions are then defined in their corresponding header and cpp files.

1.1. Colour Correction and Simple Per-pixel Modifiers:

Greyscale: create a new array in size of width*height if the input image has multiple channels. Then calculate the pixel value using for loop with the luminance definition. Then set the image data to the created array and change channel number to 1.

Brightness: The brightness takes an optional input, when the input has been given, all pixels would add the optional input, and boundaries would be set if the pixel values ever exceed 255 or -255. Else if the input is missing, the function would calculate average across all channels, and subtract the average by 128, then add the number got from the subtraction to all pixels in the channel.

Histogram Equalisation: Enhances image contrast by redistributing pixel intensities. It adapts based on image type—using different strategies for grayscale and colour images—by converting to HSV/HSL to boost brightness/luminance while preserving hue and saturation.

Optimisations include lookup tables and a single-pass histogram computation, achieving $O(n+256)$ complexity.

Thresholding: Converts images to clear binary (black/white) output for feature extraction. It supports multiple formats (grayscale, RGB, RGBA), applies similar colour space handling, and preserves the alpha channel when present. It validates threshold values (0–255) and uses in-place modifications with an early return, reaching $O(n)$ complexity.

Salt and Pepper: for this task, random numbers are generated to form the indices to add noise to and then add noise to the pixel values.

1.2. Blur:

Box Blur: This algorithm uses separable filtering by breaking the 2D convolution into two 1D passes: first horizontally, then vertically. Because one multiplication of two matrices can be divided into two multiplications of a vector and a matrix, if the condition holds that the multiplication of two vectors (horizontal and vertical) equals to the filter matrix. It employs a sliding window technique to efficiently compute the average for each pixel, reducing redundant calculations and significantly lowering the time complexity compared to a direct 2D kernel application.

Gaussian Blur: The algorithm designed leverages separable convolution by first generating a 1D Gaussian kernel and then applying it in two passes—first horizontally and then vertically—to efficiently achieve a 2D blur, which used the same approach implemented in the ‘**Box Blur**’. It uses a sliding window to compute weighted averages and handles boundary conditions by checking indices, reducing the time complexity compared to a full 2D convolution.

Median Blur: This median blur algorithm computes a histogram of pixel intensities for each sliding window (per row and channel) instead of sorting all values for each pixel. It initializes the histogram for the first window and then updates it incrementally—removing the left column and adding the right column as the window slides—while handling boundary conditions via mirror padding and finally finds the median from the histogram.

1.3 Sharpening Filter:

There are few factors to consider when filtering images, the padding of the image or it could be seen as boundary condition (otherwise the size of the image would be different from the original one), and the indices of the neighbour elements for calculation. The padding method here would be mirror padding, instead of changing the array, the indices are fully used to check the boundary conditions. If it is top or left, if the place < 0 , (current row or column index), it would access value in index “(-place)”, else if it is bottom or right, place $> \text{maxvalue}-1$, where maxvalue is the row number or column number, it would return value of index “(2 * maxvalue – place - 2)”. Then with appropriate indexing manipulation, the filter can be applied across all channels to get the sharpened results.

1.4 Edge Detection:

The edge detection filter task contains two steps, one is applying greyscale on the image if which has multiple channels, then applying G_x and G_y filters accordingly, then 2-norm would

be used on results of Gx and Gy to get the current pixel value. The design for greyscale has been demonstrated in the content above, thus, the idea of applying filters would be focused on this part. For image padding, the same approach of padding in sharpening filter would be used, which is mirroring extension. Then for loops are used to iterate through each pixel of the image and then calculate the productions of the matrices(filtering), and Gx and Gy would be calculated respectively and then combine them to form new pixels, which would then be stored in a new array to which the image data would be changed.

2.1 3D Filters

3D Gaussian Blur: Because the gaussian filter could be treated as the projection of gaussian distribution on a matrix, therefore, a 3D gaussian filter would be similar to that of a 2D gaussian filter. Thus, same approach in the 2D gaussian filter can be implemented, which divides that gaussian filter into 3 different vectors in different orientations, and apply these vectors multiplication using sliding window, vertically, horizontally and in the depth. This approach would prevent iterating through every pixel using a $N*N$ matrix, instead, three vectors of length 3 have been applied to reduce the time complexity of the algorithm.

3D Median Blur: The 3D median blur used similar approach that have been implemented in the 2D blur, which is using histogram to store the neighbours of the central voxel and then sort them using the idea of counting sort, which counts the number of occurrences from low values to heigh until the median have been accessed. The median projection would use similar approach to find the median values.

2.2 Orthographic Projections

For MIP and MinIP, the logic is quite similar, instead of using multiple for loops to iterate through the entire volume, only the height and width should be iterated, at each pixel, elements in the depth would be accessed consecutively until max or min has been acquired. Therefore, the procedure works in this way: Firstly, set for loops to iterate through each pixel, then access the depth pixels in coordinates with appropriate indices, when accessing the pixels, compare current pixel with proceeding pixel until max or min, then return the result image after iteration on all pixels.

For the meanAIP, instead of comparing the pixels, the function would add up the pixels together and then calculate the average.

For medianAIP, sorting algorithm would be needed. A histogram array is created from 0 to 255 (length 256), when accessing a pixel, the corresponding index would add one, and the median could be calculated by counting from the histogram. For example, if the depth is 5, adding up the histogram array from index 0 until sum is 3, the current value of the index that contributes to 3 is the median value.

2.3 Slicing

The slicing has three different orientations, XY, XZ, and YZ planes. Each orientation can be done by appropriate indexing. For example, in XZ plane, the input would be height. Then create a new Image object to hold the slicing result, in the for loop, the indices to be updated is $[(z * \text{width} + x) * c + c]$, the $(z * \text{width} + x)$ gets the pixel of number z row and column x in XZ plane,

and * c means jumping through each pixel block by its channel number, because the image is stored in the way: pixel1_channel1, pixel1_channel2, pixel1_channel3, pixel2_channel1 etc. +c corresponds to the c+1-th channel of the current pixel. Since the index has been acquired, the value to update would be volume[x, input_y_index, z, c].

Performances (compared with opencv library):

The comparisons of our project and python opencv library have been made to ensure that we are on the correct path.



The two graphs above shows part of the tests we have written, the left one is greyscale while the right one shows the edge detection algorithm. The data on the greyscale image showed that the running time of our self-implemented function has satisfying performance compared to the opencv implementation. The error is the sum of pixel-wise error between opencv generated image and the result of our project, therefore, the implementation is appropriate as it achieves nearly same result as the opencv does. However, for edge detection algorithm, our running time takes approximately ten times longer, which indicates that the performance of the edge detection algorithm remains to be improved. There might be few reasons for that, opencv is an extremely optimized library built by plenty of data scientists with much time, thus, the people and time constraints remain to be a limitation for our team to achieve the same results as opencv.

However, we know how to achieve higher performance in some aspects. Firstly, the opencv utilises parallel and multi-threaded programming as well as GPU acceleration if cuda might be available. Also, compared to convolution with the entire filter, the opencv would use vector multiplication, like the strategy we used for box and gaussian blurring, for example, the Gx filter for Sobel filter can be divided into multiplication of two vectors which are [1,2,1] vertical vector, and [-1,0,1] horizontal vector. Therefore, if given enough time, there are still more codes where we can optimize.

For projection or slicing, there are no functions in opencv thus it will not be compared here, because the implementation of python would be slower than cpp, thus only the above two functions would be demonstrated for the reason that opencv has same function which can be easily compared in terms of error or time.